JPA single-valued association

학습목표

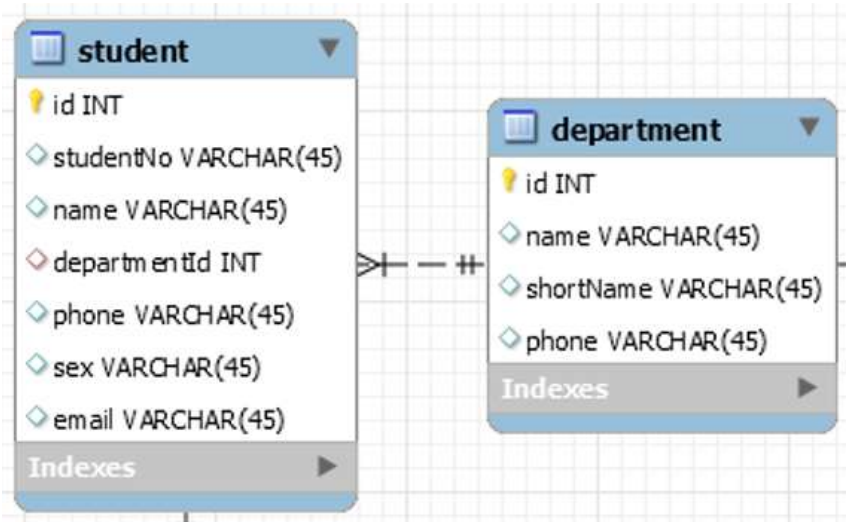
single valued association 구현
학생 목록/등록/수정/삭제 구현

목차

1. 배경지식 2
   1) Single-Valued Association 구현 2
2. jpa2 프로젝트 4
   1) 프로젝트 생성 4
   2) build.gradle 파일 수정 6
   3) application.properties 7
   4) Department 엔터티 클래스 8
   5) Student 엔터티 클래스 8
   6) DepartmentRepository 클래스 9
   7) StudentRepository 클래스 9
   8) StudentEdit 모델 클래스 10
   9) StudentController 구현 11
   10) common.css 13
   11) student/list 뷰 구현 14
   12) student/edit.html 15
   13) 실행 16

1.배경지식

1) Single-Valued Association 구현



department 와 student 관계는 1 대 다 관계이다.
student 테이블의 departmentId 필드는 소속 학과를 가르키는 외래키(foreign key) 이다.

department 와 student 관계의 구현은
외래키(departmentId)가 포함된 student 객체에 department 멤버 변수를 구현한다.

@ManyToOne 어노테이션

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;
    String name;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
```

학생 여러명이 학과 한 개에 소속되어 있다.
학생 테이블의 외래 키는 departmentId 필드이다.
학생이 속한 학과는 한 개이다.
departmentId 속성을 만들지 않음에 주목하자.

```
@ManyToOne
@JoinColumn(name = "departmentId")
  Department department
```

Student 와 Department 관계는 Many To One 이다.
그래서 Student 클래스의 department 멤버 변수에 @ManyToOne 어노테이션을 붙여야 한다.

이 관계는 departmentId 외래키로 표현된다.
student 테이블과 department 테이블을 조인할 때,
departmentId 외래키로 조인해야 한다.

```
@ManyToOne
@JoinColumn(name = "departmentId")
Department department;
```

요약

엔터티 클래스들 사이의 1 대 다 관계를 구현할 때,
외래키를 포함하는 엔터티 클래스에 @ManyToOne 어노테이션을 사용하여,
상대측 엔터티 객체 한 개를 구현한다.
그리고 외래키 속성은 생략한다.

Student 엔터티 클래스에 departmentId 외래키가 있으므로

다음과 같은 single valued association Department 속성을 구현 한다.
그리고 departmentId 속성을 또 만들지 않아야한다.

```
@ManyToOne
@JoinColumn(name = "departmentId")
Department department;
```

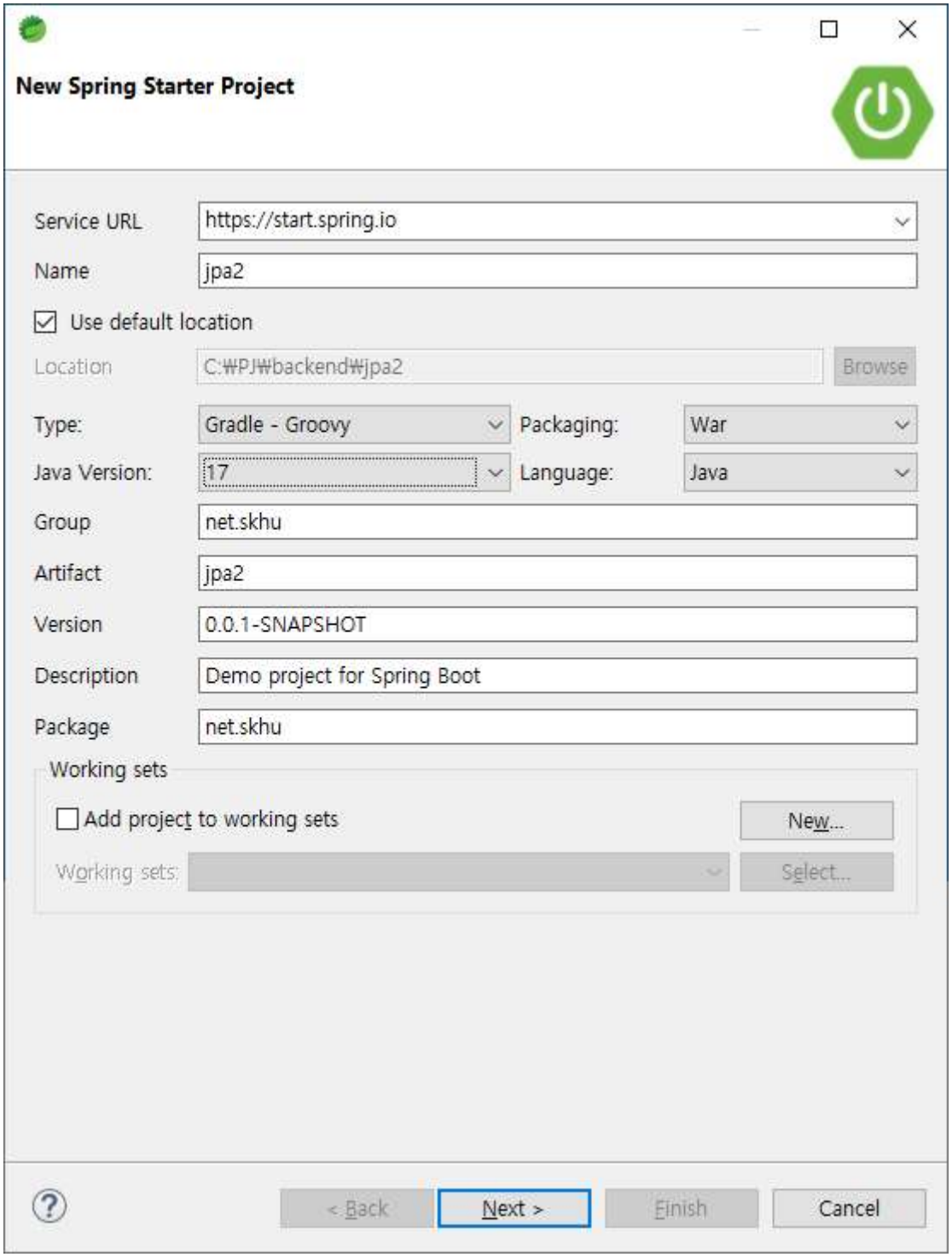
위와 같이 departmentId 외래키 대신 Department department 멤버 변수를
Student 엔터티 클래스에 정의하면 다음과 같은 일이 자동으로 일어난다.

JPA 엔진이, student 테이블에서 학생 레코드를 조회해서 Student 엔터티 객체를 생성할 때,
Student 객체의 department 속성에 학과 정보도 채워준다.

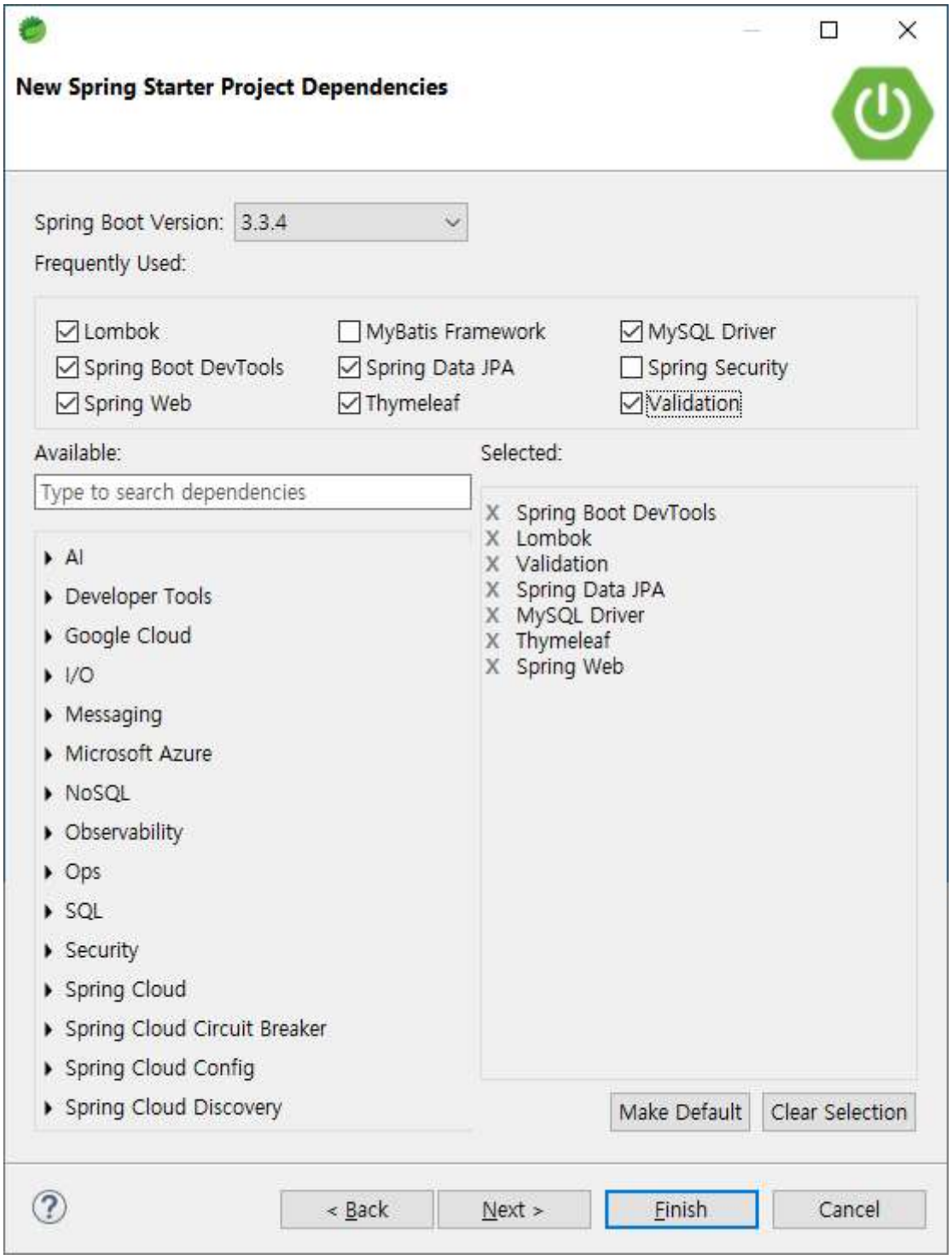
즉 JPA 엔진이, student 테이블에서 학생 레코드를 조회해서 Student 엔터티 객체를 생성할 때,
그 학생의 departmentId 외래키로 department 테이블도 조회해서,
Department 엔터티 객체를 생성하여,
Student 객체의 department 속성에 Department 객체도 채워준다.

2. jpa2 프로젝트

1) 프로젝트 생성



Name	jpa2
Packaing	war
Group	net.skhu
Artifact	jpa1
Package	net.skhu



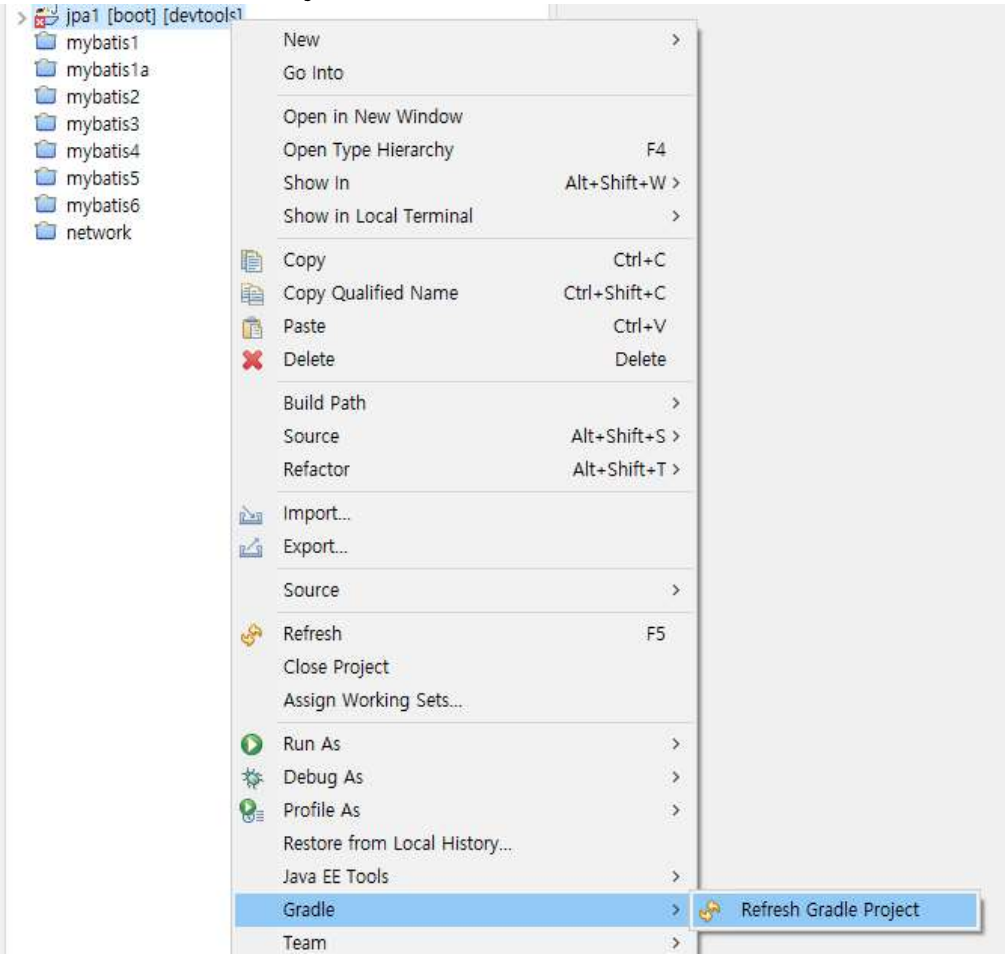

```
Spring Boot DevTools
Lombok
Validation
Spring Data JPA
MySQL Driver
Thymeleaf
Spring Web
```

2) build.gradle 파일 수정

```
dependencies {
  ... 생략 ...
    implementation 'org.modelmapper:modelmapper:3.2.5'
}
```

gradle 프로젝트의 설정 파일인 build.gradle 파일에
  modelmapper maven dependency 한 줄을 추가

Referesh Gradle Project



build.gradle 파일을 수정한 후에는 반드시 Refresh Gradle Project 해야 한다.

3) application.properties

```
spring.application.name=jpa2
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/student2?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=user1
spring.datasource.password=skhuA+4.5
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
logging.level.net.skhu=DEBUG
server.port=8088
```

4) Department 엔터티 클래스

src/main/java/net/skhu/entity/Department.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String name;
    String shortName;
    String phone;
}
```

5) Student 엔터티 클래스

src/main/java/net/skhu/entity/Student.java

```java
package net.skhu.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Data;

@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    int id;

    String studentNo;
    String name;
    String phone;
    String sex;
    String email;

    @ManyToOne
    @JoinColumn(name = "departmentId")
    Department department;
}
```

6) DepartmentRepository 클래스

src/main/java/net/skhu/repository/DepartmentRepository.java

```
1  package net.skhu.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import net.skhu.entity.Department;
5
6  public interface DepartmentRepository extends JpaRepository<Department, Integer>  {
7
8  }
```

7) StudentRepository 클래스

src/main/java/net/skhu/repository/StudentRepository.java

```
1  package net.skhu.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import net.skhu.entity.Student;
5
6  public interface StudentRepository extends JpaRepository<Student, Integer>  {
7
8  }
```

8) StudentEdit 모델 클래스

src/main/java/net/skhu/model/StudentEdit.java

```java
package net.skhu.model;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class StudentEdit {
    int id;

    @NotEmpty @NotBlank
    @Size(min=8, max=12)
    String studentNo;

    @NotEmpty @NotBlank
    @Size(min=2, max=20)
    String name;

    @NotEmpty @NotBlank
    @Pattern(regexp="010-[0-9]{3,4}-[0-9]{4}",
             message="휴대폰 번호를 입력하세요 예:010-000-0000")
    String phone;

    @NotEmpty @Email
    String email;

    @NotEmpty @NotBlank
    @Pattern(regexp="남|여", message="남, 여 중 하나를 입력하세요.")
    String sex;

    @Min(value=1, message="학과를 선택하세요.")
    int departmentId;
}
```

9) StudentController 구현

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import java.util.List;
import java.util.Optional;
import org.modelmapper.ModelMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;
import jakarta.validation.Valid;
import lombok.extern.slf4j.Slf4j;
import net.skhu.entity.Student;
import net.skhu.model.StudentEdit;
import net.skhu.repository.DepartmentRepository;
import net.skhu.repository.StudentRepository;

@Controller
@Slf4j
@RequestMapping("student")
public class StudentController {

    @Autowired StudentRepository studentRepository;
    @Autowired DepartmentRepository departmentRepository;
    ModelMapper modelMapper = new ModelMapper();

    @GetMapping("list")
    public String list(Model model) {
        List<Student> students = studentRepository.findAll();
        model.addAttribute("students", students);
        return "student/list";
    }

    @GetMapping("edit")
    public String edit(Model model, @RequestParam("id") int id) {
        Optional<Student> student = studentRepository.findById(id);
        if (student.isEmpty())
            throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        StudentEdit studentEdit = modelMapper.map(student.get(), StudentEdit.class);
        model.addAttribute("studentEdit", studentEdit);
        model.addAttribute("departments", departmentRepository.findAll());
        return "student/edit";
    }

    @PostMapping(value="edit", params="cmd=save")
    public String edit(Model model, @Valid StudentEdit studentEdit, BindingResult bindingResult) {
        try {
            log.debug(studentEdit.toString());
            if (bindingResult.hasErrors())
                throw new Exception("저장할 수 없습니다");

            Student student = modelMapper.map(studentEdit, Student.class);
            studentRepository.save(student);
            return "redirect:list";
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            model.addAttribute("departments", departmentRepository.findAll());
            return "student/edit";
        }
    }

    @PostMapping(value="edit", params="cmd=delete")
    public String delete(Model model, @Valid StudentEdit studentEdit, BindingResult bindingResult) {
        try {
            studentRepository.deleteById(studentEdit.getId());
            return "redirect:list";
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            model.addAttribute("departments", departmentRepository.findAll());
            return "student/edit";
        }
    }

    @GetMapping("create")
    public String create(Model model) {
        StudentEdit studentEdit = new StudentEdit();
```

```
84          model.addAttribute("studentEdit", studentEdit);
85          model.addAttribute("departments", departmentRepository.findAll());
86          return "student/edit";
87      }
88
89      @PostMapping("create")
90      public String create(Model model, @Valid StudentEdit studentEdit, BindingResult bindingResult) {
91          try {
92              log.debug(studentEdit.toString());
93              if (bindingResult.hasErrors())
94                  throw new Exception("저장할 수 없습니다");
95
96              Student student = modelMapper.map(studentEdit, Student.class);
97              studentRepository.save(student);
98              return "redirect:list";
99          }
100         catch (Exception ex) {
101             bindingResult.reject("", null, ex.getMessage());
102             model.addAttribute("departments", departmentRepository.findAll());
103             return "student/edit";
104         }
105     }
106 }
```

```
Student student = modelMapper.map(studentEdit, Student.class);
```

studentEdit 객체에는 departmentId 속성이 있다.

그런데 Student 엔터티 객체에는 departmentId 속성이 없고
single-valued association 속성인 department 객체 속성이 있다.
이 department 객체에 id 속성이 있다.

위 코드에서 model mapper는
studentEdit 객체의 departmentId 속성 값을
student 엔터티 객체의 department 속성의 id 속성에 값을 자동으로 채워준다.

즉 studentEdit.departmentId 속성값이 student.department.id 속성에 자동으로 채워준다.

student.departmentId (속성 없음)
student.department (속성 있고)
student.department.id (속성 있음)

studentEdit.departmentId (속성 있음)

student.departmentId 속성은 없지만
대신 student.department.id 속성이 있으므로,
model mapper가 studentEdit.departmentId 속성값을 student.department.id 속성에
채워줄 정도로 똘똘하다.

10) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }

div.pagination table { border-collapse: collapse; margin-top: 1em; }
div.pagination table td { border: 1px solid #ccc; text-align: center; }
div.pagination table td.active { background-color: #48f; }
div.pagination table a { text-decoration: none; color: blue; display: inline-block;
                         width: 25px; padding: 6px; }
div.pagination table td.active a { color: white; }
```

11) student/list 뷰 구현

src/main/resources/templates/student/list.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
    a.btn { float:right; margin-top: -37px; }
  </style>
</head>
<body>
<div class="container">
  <h1>학생목록</h1>
  <a href="create" class="btn">학생 등록</a>
  <table class="list">
    <thead>
      <tr>
        <th>id</th>
        <th>학번</th>
        <th>이름</th>
        <th>학과</th>
        <th>전화</th>
        <th>성별</th>
        <th>이메일</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="st : ${ students }">
        <td th:text="${ st.id }"></td>
        <td>
          <a th:text="${ st.studentNo }" th:href="@{ edit(id=${ st.id }) }"></a>
        </td>
        <td th:text="${ st.name }"></td>
        <td th:text="${ st.department.name }"></td>
        <td th:text="${ st.phone }"></td>
        <td th:text="${ st.sex }"></td>
        <td th:text="${ st.email }"></td>
      </tr>
    </tbody>
  </table>
</div>
</body>
</html>
```

"${ st.department.name }"
   list.html 뷰에 전달된 학생 목록에서 학생 객체는
      Student 엔터티 클래스 객체이다.
      Student 엔터티 클래스에는 departmentName 속성이 없고
      department 속성이 있다.
      이 department 속성은 Department 객체이고
      Department 객체에는 name 속성이 있다.

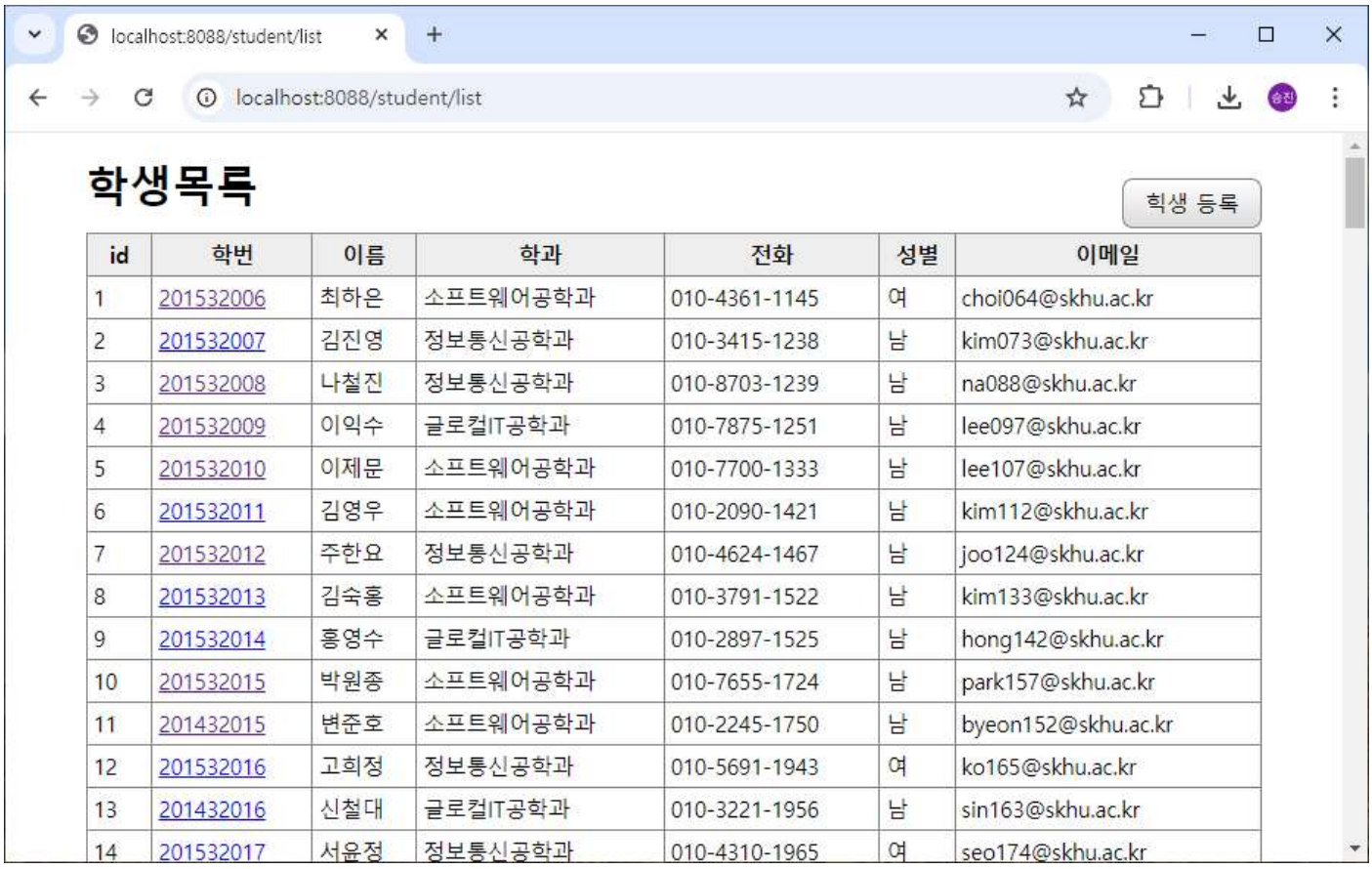
이전 mybatis 프로젝트 강의 예제에서
  student DTO 클래스에는 departmentName 속성이 있었지만,
JPA 프로젝트의 student 엔터티 클래스에는 departmentName 속성이 없다.
대신 student.department.name 속성이 있다. (department 객체의 name 속성)

12) student/edit.html

src/main/resources/templates/student/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" th:href="@{/common.css}" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button:nth-of-type(1) { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${studentEdit}" class="shadow">
    <h1 th:if="${studentEdit.id > 0}">학생 수정</h1>
    <h1 th:if="${studentEdit.id == 0}">학생 등록</h1>
    <div class="row">
      <label>학번:</label>
      <input type="text" th:field="*{studentNo}" />
      <div class="error" th:errors="*{studentNo}"></div>
    </div>
    <div class="row">
      <label>이름:</label>
      <input type="text" th:field="*{name}" />
      <div class="error" th:errors="*{name}"></div>
    </div>
    <div class="row">
      <label>학과:</label>
      <select th:field="*{departmentId}">
         <option th:each="d : ${departments}"
                th:value="${d.id}" th:text="${d.name}"></option>
      </select>
      <div class="error" th:errors="*{departmentId}"></div>
    </div>
    <div class="row">
      <label>성별:</label>
      <label><input type="radio" th:field="*{sex}" value="남" />남자</label>
      <label><input type="radio" th:field="*{sex}" value="여" />여자</label>
      <div class="error" th:errors="*{sex}"></div>
    </div>
    <div class="row">
      <label>전화:</label>
      <input type="text" th:field="*{phone}" />
      <div class="error" th:errors="*{phone}"></div>
    </div>
    <div class="row">
      <label>메일:</label>
      <input type="text" th:field="*{email}" />
      <div class="error" th:errors="*{email}"></div>
    </div>
    <hr />
    <button type="submit" class="btn"
             name="cmd" value="save">저장</button>
    <button type="submit" class="btn"
             th:if="${ studentEdit.id > 0 }"
             onclick="return confirm('삭제하시겠습니까?')"
             name="cmd" value="delete">삭제</button>
    <a href="list" class="btn">목록으로</a>
    <span class="error" th:errors="${studentEdit}"></span>
  </form>
</div>
</body>
</html>
```

13) 실행



id	학번	이름	학과	전화	성별	이메일
1	201532006	최하은	소프트웨어공학과	010-4361-1145	여	choi064@skhu.ac.kr
2	201532007	김진영	정보통신공학과	010-3415-1238	남	kim073@skhu.ac.kr
3	201532008	나철진	정보통신공학과	010-8703-1239	남	na088@skhu.ac.kr
4	201532009	이익수	글로컬IT공학과	010-7875-1251	남	lee097@skhu.ac.kr
5	201532010	이제문	소프트웨어공학과	010-7700-1333	남	lee107@skhu.ac.kr
6	201532011	김영우	소프트웨어공학과	010-2090-1421	남	kim112@skhu.ac.kr
7	201532012	주한요	정보통신공학과	010-4624-1467	남	joo124@skhu.ac.kr
8	201532013	김숙홍	소프트웨어공학과	010-3791-1522	남	kim133@skhu.ac.kr
9	201532014	홍영수	글로컬IT공학과	010-2897-1525	남	hong142@skhu.ac.kr
10	201532015	박원종	소프트웨어공학과	010-7655-1724	남	park157@skhu.ac.kr
11	201432015	변준호	소프트웨어공학과	010-2245-1750	남	byeon152@skhu.ac.kr
12	201532016	고희정	정보통신공학과	010-5691-1943	여	ko165@skhu.ac.kr
13	201432016	신철대	글로컬IT공학과	010-3221-1956	남	sin163@skhu.ac.kr
14	201532017	서윤정	정보통신공학과	010-4310-1965	여	seo174@skhu.ac.kr